

一、資料介紹

問題：由糖尿病資料集建立預測模型來識別有糖尿病風險的個體。

資料來源：Diabetes Raw Dataset 訓練集有 800 筆資料，測試集有 200 筆資料。

(<https://www.kaggle.com/datasets/yash9091/diabetes-raw-dataset>)

欄位介紹：

(1) ID：識別碼

(2) Pregnancies：懷孕次數

(3) Glucose：口服葡萄糖耐受試驗中 2 小時內的血漿葡萄糖濃度

(4) BloodPressure：血壓，舒張壓（mmHg）

(5) SkinThickness：三頭肌皮下脂肪厚度（mm/cm）

(6) Insulin：胰島素，兩小時血清胰島素濃度

(7) BMI：身體質量指數（weight in kg / height in m²）

(8) DiabetesPedigreeFunction：糖尿病的遺傳評分

(9) Age：年齡

(10) Outcome：是否有糖尿病，No 為否，Yes 為是

Features：Pregnancies、Glucose、BloodPressure、SkinThickness、Insulin、
BMI、DiabetesPedigreeFunction、Age

Label：Outcome

二、資料分析與清理

匯入檔案，先用 `head()` 看一下資料的樣子。

```
import pandas as pd
train_df = pd.read_csv('Diabetes-train.csv')
train_df.head()
```

0.0s

	ID	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	343	1	0.0	68	35	No	32.0	0.389	22	No
1	496	6	166.0	74	No	No	26.6	0.304	66	No
2	1429	10	162.0	84	No	No	27.7	0.182	54	No
3	1722	0	84.0	82	31	125	38.2	0.233	23	No
4	1614	12	140.0	82	43	325	39.2	0.528	58	Yes

發現特徵 `Glucose` 和 `SkinThickness` 中有缺失值，而且特徵 `BloodPressure`、`SkinThickness`、`Insulin` 和 `Outcome` 的資料型態為類別，其他特徵型態為整數或浮點數。

```
train_df.info()
```

0.0s

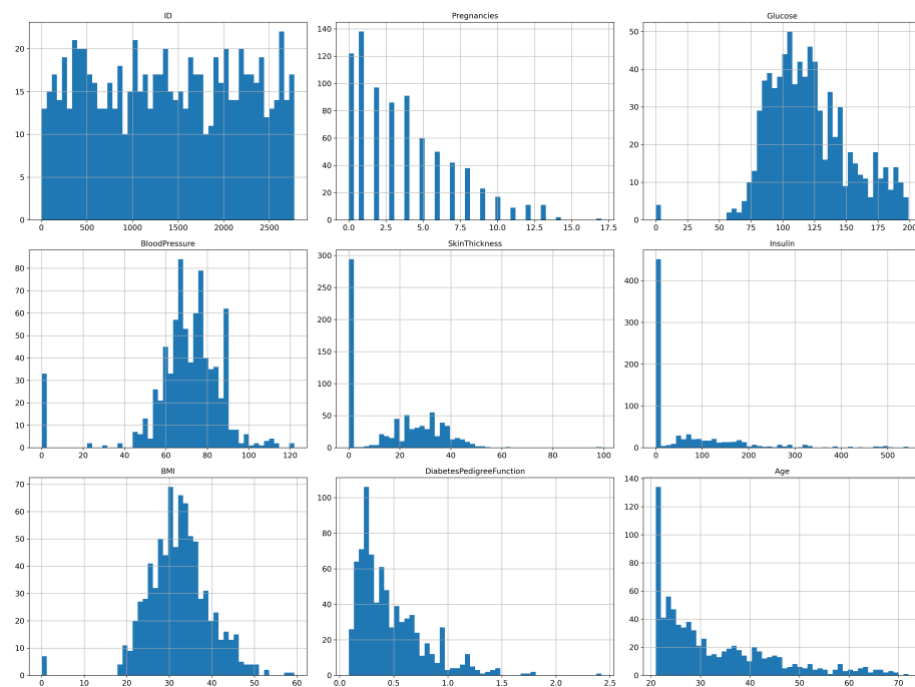
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 10 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   ID                   800 non-null   int64
1   Pregnancies          800 non-null   int64
2   Glucose              799 non-null   float64
3   BloodPressure        800 non-null   object
4   SkinThickness        799 non-null   object
5   Insulin              800 non-null   object
6   BMI                  800 non-null   float64
7   DiabetesPedigreeFunction 800 non-null   float64
8   Age                  800 non-null   int64
9   Outcome              800 non-null   object
dtypes: float64(3), int64(3), object(4)
memory usage: 62.6+ KB
```

移除缺失值後，用 `astype('float')` 將特徵 `BloodPressure`、`SkinThickness` 和 `Insulin` 轉成數值型態後，再用 `describe()` 呈現出資料的最大最小值與平均數等資訊，由下圖可以發現 `BloodPressure` 和 `BMI` 中最小值都為 0。

```
train_df.describe()
```

	ID	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
count	798.000000	798.000000	798.000000	798.000000	798.000000	798.000000	798.000000	798.000000	798.000000
mean	1390.008772	3.694236	120.917293	69.854637	18.158772	63.817043	32.272807	0.467989	33.047619
std	802.515323	3.252524	32.110196	19.100708	16.119596	100.789090	7.287515	0.309133	11.605832
min	4.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.084000	21.000000
25%	688.250000	1.000000	98.000000	64.000000	0.000000	0.000000	27.700000	0.241000	24.000000
50%	1378.500000	3.000000	117.000000	72.000000	19.000000	0.000000	32.300000	0.381500	29.000000
75%	2084.750000	6.000000	140.000000	80.000000	32.000000	103.750000	36.500000	0.619000	40.000000
max	2774.000000	17.000000	199.000000	122.000000	99.000000	545.000000	59.400000	2.420000	72.000000

利用`hist()`畫出直方圖，可以看出每個特徵裡資料的分布狀況。



因為 BMI 值是體重/身高平方，值為 0 時為異常值，所以將它去除，並確定去除後資料大小是否正確。

```
condition = train_df['BMI']==0
train_df[condition]
```

✓ 0.0s

	ID	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
29	1140	0	118.0	64.0	23.0	89.0	0.0	1.731	21	No
174	2541	5	136.0	82.0	0.0	0.0	0.0	0.640	69	No
385	914	0	102.0	75.0	23.0	0.0	0.0	0.572	21	No
409	1665	0	94.0	0.0	0.0	0.0	0.0	0.256	25	No
426	146	0	102.0	75.0	23.0	0.0	0.0	0.572	21	No
570	1291	6	114.0	0.0	0.0	0.0	0.0	0.189	26	No
627	523	6	114.0	0.0	0.0	0.0	0.0	0.189	26	No

```
train_df = train_df.loc[~(train_df['BMI'] == 0)]
train_df.shape
```

✓ 0.0s

(791, 10)

BloodPressure 為 0 的資料有 30 筆，一樣將其挑出並去除，並確認資料大小是否正確。

```
condition = train_df['BloodPressure']==0
train_df[condition]
```

✓ 0.0s

Outputs are collapsed ...

```
train_df = train_df.loc[~(train_df['BloodPressure'] == 0)]
train_df.shape
```

✓ 0.0s

(761, 10)

將 Outcome 的 Yes/No 換成 1/0，並展示資料清理後的前五項。

```
1 train_df = train_df.replace(to_replace=["No"], value="0")
2 train_df = train_df.replace(to_replace=["Yes"], value="1")
3 train_df.head()
```

✓ 0.0s

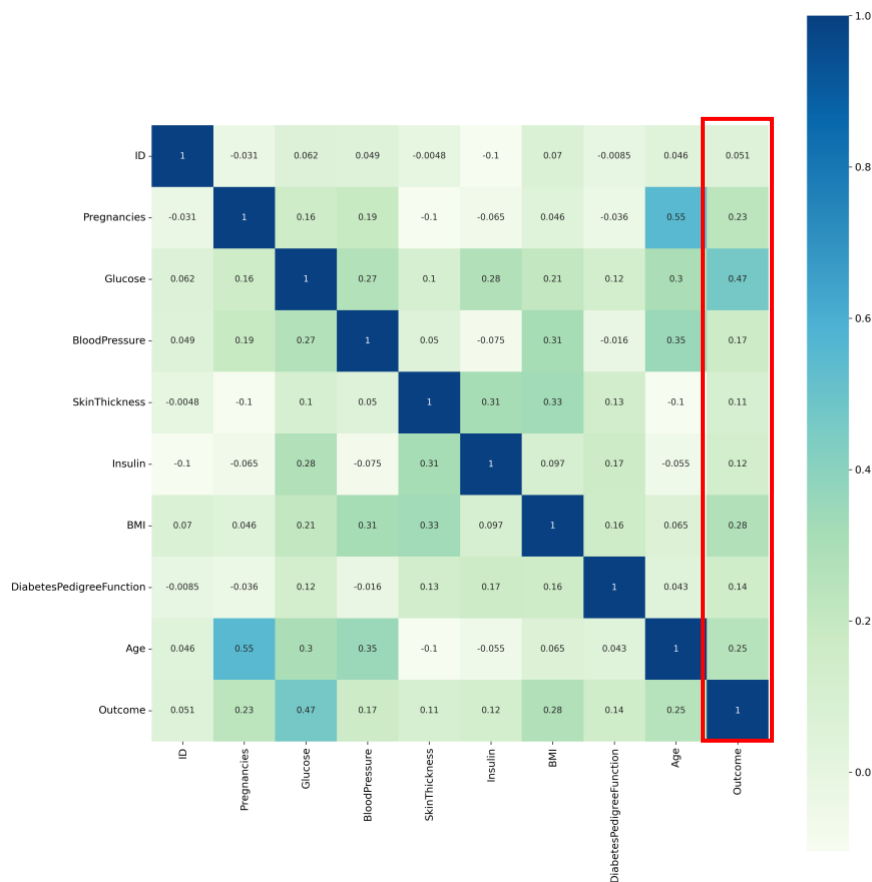
	ID	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	343	1	0.0	68.0	35.0	0.0	32.0	0.389	22	0
1	496	6	166.0	74.0	0.0	0.0	26.6	0.304	66	0
2	1429	10	162.0	84.0	0.0	0.0	27.7	0.182	54	0
3	1722	0	84.0	82.0	31.0	125.0	38.2	0.233	23	0
4	1614	12	140.0	82.0	43.0	325.0	39.2	0.528	58	1

三、相關性矩陣

用 `corr()` 得到特徵與 Outcome 還有特徵與特徵間的相關係數，之後用 `seaborn` 畫成相關係數圖，從圖中發現 ID 與 Outcome 的相關係數只有 0.051，覺得太小了決定去除這個特徵，其他特徵與 Outcome 的相關係數介於 0.1~0.5 間。

```
1 import seaborn as sns
2 corr_matrix = train_df.corr()
3 plt.figure(figsize=(15,15))
4 ax = sns.heatmap(corr_matrix, square=True, annot=True, cmap='GnBu')
5 save_fig("corr_matrix_plots")
6 plt.show()
```

✓ 1.3s



去除 ID 後的資料。

```
train_df = train_df.drop('ID', axis=1)
train_df.head()
```

✓ 0.0s

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	1	0.0	68.0	35.0	0.0	32.0	0.389	22	0
1	6	166.0	74.0	0.0	0.0	26.6	0.304	66	0
2	10	162.0	84.0	0.0	0.0	27.7	0.182	54	0
3	0	84.0	82.0	31.0	125.0	38.2	0.233	23	0
4	12	140.0	82.0	43.0	325.0	39.2	0.528	58	1

四、模型訓練與選擇

將 train_df 切割 80% 做訓練集，20% 做預測集，並確定資料的大小是否正確。

```
1 from sklearn.model_selection import train_test_split
2 X = train_df.drop('Outcome', axis=1)
3 y = train_df['Outcome'].astype(int)
4 train_X, test_X, train_y, test_y = train_test_split(X, y, test_size=0.2, random_state=42)
5 print(train_X.shape, train_y.shape, test_X.shape, test_y.shape)
```

✓ 0.0s

(608, 8) (608,) (153, 8) (153,)

定義 threshold_predictions()，將預測結果轉成 0/1。

```
def threshold_predictions(predictions, threshold=0.5):
    """
    將預測結果轉換為二元分類的0或1，根據給定的閾值。

    Parameters:
    - predictions: 預測結果的列表或陣列
    - threshold: 閾值，預設為0.5

    Returns:
    - binary_predictions: 二元分類的0或1列表或陣列
    """
    binary_predictions = [1 if pred > threshold else 0 for pred in predictions]
    return binary_predictions
```

1 ✓ 0.0s

利用線性迴歸得到的 MSE 與 MAE 還有準確率。

```
#LinearRegression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, accuracy_score

lin_reg = LinearRegression()
lin_reg.fit(train_X, train_y)
#pred1 = lin_reg.predict(test_X)
pred1 = threshold_predictions(lin_reg.predict(test_X))

lin_mse = mean_squared_error(test_y, pred1)
lin_rmse = np.sqrt(lin_mse)
lin_mae = mean_absolute_error(test_y, pred1)

print('MSE:', lin_mse)
print('MAE:', lin_mae)
print('Accuracy:', accuracy_score(test_y, pred1))
```

✓ 0.2s

MSE: 0.20915032679738563
MAE: 0.20915032679738563
Accuracy: 0.7908496732026143

利用決策樹得到的 MSE 與 MAE 還有準確率。

```
#決策樹
from sklearn.tree import DecisionTreeRegressor
tree_reg = DecisionTreeRegressor(random_state=42)
tree_reg.fit(train_X, train_y)
pred2 = tree_reg.predict(test_X)

dt_mse = mean_squared_error(test_y, pred2)
dt_rmse = np.sqrt(dt_mse)
dt_mae = mean_absolute_error(test_y, pred2)

print('MSE:',dt_mse)
print('MAE:',dt_mae)
print('Accuracy:',accuracy_score(test_y, pred2))
```

MSE: 0.16339869281045752
MAE: 0.16339869281045752
Accuracy: 0.8366013071895425

利用隨機森林得到的 MSE 與 MAE 還有準確率。

```
#隨機森林
from sklearn.ensemble import RandomForestRegressor
forest_reg = RandomForestRegressor(n_estimators=100, random_state=42)
forest_reg = forest_reg.fit(train_X, train_y)
#pred3 = forest_reg.predict(test_X)
pred3 = threshold_predictions(forest_reg.predict(test_X))

rf_mse = mean_squared_error(test_y, pred3)
rf_rmse = np.sqrt(rf_mse)
rf_mae = mean_absolute_error(test_y, pred3)

print('MSE:',rf_mse)
print('MAE:',rf_mae)
print('Accuracy:',accuracy_score(test_y, pred3))
```

MSE: 0.11764705882352941
MAE: 0.11764705882352941
Accuracy: 0.8823529411764706

利用支援向量機得到的 MSE 與 MAE 還有準確率。

```
#支援向量機
from sklearn.svm import SVR
svm_reg = SVR(kernel="linear")
#svm_reg = SVR(kernel="rbf") #accuracy:0.7777777
svm_reg.fit(train_X, train_y)
#pred4 = svm_reg.predict(test_X)
pred4 = threshold_predictions(svm_reg.predict(test_X))

rf_mse = mean_squared_error(test_y, pred4)
rf_rmse = np.sqrt(rf_mse)
rf_mae = mean_absolute_error(test_y, pred4)

print('MSE:',rf_mse)
print('MAE:',rf_mae)
print('Accuracy:',accuracy_score(test_y, pred4))
```

✓ 5.7s

MSE: 0.20261437908496732
MAE: 0.20261437908496732
Accuracy: 0.7973856209150327

利用感知器得到的 MSE 與 MAE 還有準確率。

```
#感知器
from sklearn.linear_model import Perceptron
per = Perceptron(max_iter=1000, tol=1e-3, random_state=42)
per.fit(train_X, train_y)
pred5 = per.predict(test_X)

rf_mse = mean_squared_error(test_y, pred5)
rf_rmse = np.sqrt(rf_mse)
rf_mae = mean_absolute_error(test_y, pred5)

print('MSE:', rf_mse)
print('MAE:', rf_mae)
print('Accuracy:', accuracy_score(test_y, pred5))
```

MSE: 0.2875816993464052
MAE: 0.2875816993464052
Accuracy: 0.7124183006535948

利用神經網路得到的 MSE 與 MAE 還有準確率。

```
#神經網路
from sklearn.neural_network import MLPRegressor
mlp_reg = MLPRegressor(hidden_layer_sizes=(100,100), max_iter=1000, tol=1e-3, random_state=42)
mlp_reg.fit(train_X, train_y)
#pred6 = mlp_reg.predict(test_X)
pred6 = threshold_predictions(mlp_reg.predict(test_X))

rf_mse = mean_squared_error(test_y, pred6)
rf_rmse = np.sqrt(rf_mse)
rf_mae = mean_absolute_error(test_y, pred6)

print('MSE:', rf_mse)
print('MAE:', rf_mae)
print('Accuracy:', accuracy_score(test_y, pred6))
```

✓ 3.8s

MSE: 0.26143790849673204
MAE: 0.26143790849673204
Accuracy: 0.738562091503268

Cross_val_score :

定義 display_scores()

```
from sklearn.model_selection import cross_val_score

def display_scores(scores):
    print("Scores:", scores)
    print("Mean:", scores.mean())
    print("Standard deviation:", scores.std())
```

✓ 0.0s

線性迴歸的 mean score 為 0.408478

```
#線性回歸
lin_scores = cross_val_score(lin_reg, train_X, train_y,
                             scoring="neg_mean_squared_error", cv=10)
lin_rmse_scores = np.sqrt(-lin_scores)
display_scores(lin_rmse_scores)
```

✓ 0.0s

Scores: [0.42485785 0.42392608 0.4121922 0.38616938 0.44332661 0.39282654
0.42159919 0.39210324 0.43259814 0.35518517]
Mean: 0.4084784405396819
Standard deviation: 0.02517937979741639

決策樹的 mean score 為 0.397860

```
#決策樹
tree_scores = cross_val_score(tree_reg, train_X, train_y,
                               scoring="neg_mean_squared_error", cv=10)
tree_rmse_scores = np.sqrt(-tree_scores)
display_scores(tree_rmse_scores)
```

✓ 0.0s

Scores: [0.31362502 0.40488817 0.40488817 0.28629917 0.46164354 0.42465029
0.40488817 0.52790958 0.34156503 0.40824829]
Mean: 0.39786054082662875
Standard deviation: 0.06683227434909378

隨機森林的 mean score 為 0.316239

支援向量機的 mean score 為 0.422273

```
#隨機森林
rf_scores = cross_val_score(forest_reg, train_X, train_y,
                             scoring="neg_mean_squared_error", cv=10)
rf_rmse_scores = np.sqrt(-rf_scores)
display_scores(rf_rmse_scores)
```

✓ 1.0s

Scores: [0.33196632 0.30761377 0.27590091 0.26490332 0.37746784 0.34413589
0.33206507 0.3274842 0.30435177 0.29650745]
Mean: 0.3162396535609366
Standard deviation: 0.03165284000980185

```
#支援向量機
svm_scores = cross_val_score(svm_reg, train_X, train_y,
                              scoring="neg_mean_squared_error", cv=10)
svm_rmse_scores = np.sqrt(-svm_scores)
display_scores(svm_rmse_scores)
```

✓ 40.7s

Scores: [0.4378913 0.44440415 0.39978524 0.40771018 0.469114 0.40029049
0.44419696 0.39541233 0.45673965 0.3671886]
Mean: 0.4222732906219184
Standard deviation: 0.03090825351903143

感知器的 mean score 為 0.645707

```
#感知器
per_scores = cross_val_score(per, train_X, train_y,
                              scoring="neg_mean_squared_error", cv=10)
per_rmse_scores = np.sqrt(-per_scores)
display_scores(per_rmse_scores)
```

✓ 0.0s

Scores: [0.70128687 0.70128687 0.73551588 0.79959006 0.54321448 0.52790958
0.57259833 0.55809982 0.78528127 0.53229065]
Mean: 0.6457073807942727
Standard deviation: 0.10376365654981037

感知器的 mean score 為 0.517348


```
#神經網路
mlp_scores = cross_val_score(mlp_reg, train_X, train_y,
                              scoring="neg_mean_squared_error", cv=10)
mlp_rmse_scores = np.sqrt(-mlp_scores)
display_scores(mlp_rmse_scores)
```

✓ 3.8s

Scores: [0.49734783 0.69299411 0.46176742 0.50080182 0.48970221 0.52742263
0.4419089 0.50152727 0.45527875 0.60473411]
Mean: 0.5173485042417509
Standard deviation: 0.07273967550091401

由上面的 mean score 與前面的 accuracy 來做判斷，我會選擇隨機森林模型，因為它的 mean score 最小而且 accuracy 最大。

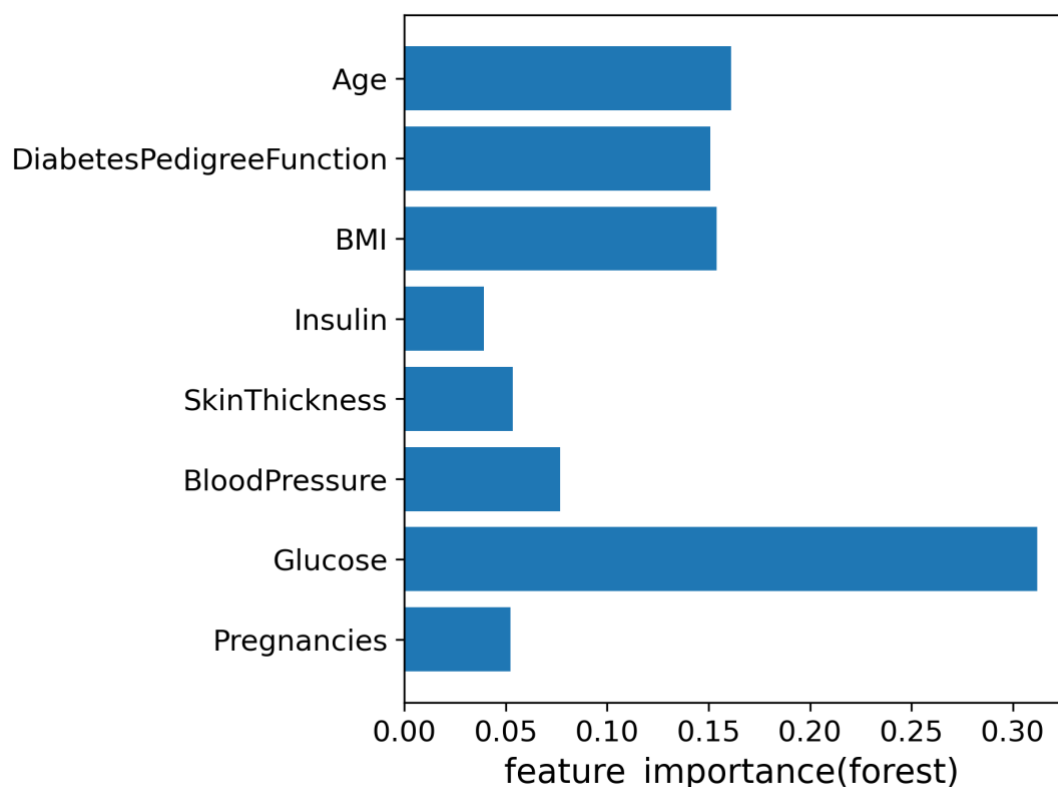
五、重要特徵

利用.feature_importances_提取出重要特徵，並繪製成圖表。

```
# 獲取特徵重要性
feature_importance_forest = forest_reg.feature_importances_

# 創建條形圖
plt.barh(range(len(feature_importance_forest)), feature_importance_forest, align='center')
plt.xticks(range(len(feature_importance_forest)), feature_names) # 假設 feature_names 是特徵名稱
plt.xlabel('feature_importance(forest) ')
save_fig("feature_importance(forest)")
plt.show()
```

由下圖可以看到隨機森林模型中選出影響力最高的特徵為 Glucose，接下來分別為 Age、DiabetesPedigreeFunction 和 BMI。



Glucose 是葡萄糖測試兩小時內的血漿葡萄糖濃度，作為判斷是否有糖尿病的重要特徵我覺得蠻合理的，另外年齡增大、肥胖（BMI 過高）和有家族病史皆有可能增加罹患糖尿病的風險，所以 Age、BMI 和 DiabetesPedigreeFunction 為有影響力的特徵也蠻符合現實情況的。

六、預測結果

讀取檔案，將類別特徵轉成數值形式。

```
test = pd.read_csv('Diabetes-test.csv')

test[['BloodPressure', 'SkinThickness', 'Insulin']] = test[['BloodPressure', 'SkinThickness', 'Insulin']].replace('No', 0)
test['BloodPressure'] = test['BloodPressure'].replace("Don't Know", 0)
test['BloodPressure'] = test['BloodPressure'].replace("4No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("2No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("4No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("1No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("3No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("2.3cm", 2.3)
test['Insulin'] = test['Insulin'].replace("5No", 0)
test['Insulin'] = test['Insulin'].replace("13No", 0)
test['Insulin'] = test['Insulin'].replace("11No", 0)
test['Insulin'] = test['Insulin'].replace("13No", 0)
test['Insulin'] = test['Insulin'].replace("12No", 0)
test['Insulin'] = test['Insulin'].replace("16No", 0)
test['Insulin'] = test['Insulin'].replace("21No", 0)
test['Insulin'] = test['Insulin'].replace("15No", 0)
test['Insulin'] = test['Insulin'].replace("18No", 0)
test['Insulin'] = test['Insulin'].replace("14No", 0)
test[['BloodPressure', 'SkinThickness', 'Insulin']] = test[['BloodPressure', 'SkinThickness', 'Insulin']].astype('float')
```

去除特徵 ID，將有缺失值的地方填入 0，然後用隨機森林模型預測出 y_pred，再把 y_pred 加入 output 中並轉存為 csv 檔。

```
test = test.drop('ID', axis=1)

test = test.fillna(0)

output = pd.DataFrame()
y_pred = threshold_predictions(forest_reg.predict(test))
output['Outcome'] = y_pred
output.to_csv('Diabetes-output.csv')
```

預測結果前五項。

```
Diabetes-output.head()
```

	Outcome
0	0
1	0
2	0
3	0
4	1

Diabetes-output.csv

(https://drive.google.com/file/d/1WBkF31J_9JEGXCScKWE4GHYla8qDx

[U_G/view?usp=sharing](#))

七、Discission & Conclusion

在 training data 裡面把特徵 BloodPressure、SkinThickness、Insulin 轉成數值資料時卡了很久，因為看前五項時以為裡面只有 No 跟數字，想說只要把 No 用 0 取代掉就可以了，結果還是一直跳出錯誤，後來發現那三個特徵裡面 No 的標示有超多種（如下圖），只好一個一個挑出來改，最後終於成功把三個特徵都轉成數值型態，沒想到 testing data 裡面也是一樣，也一樣一個一個挑出來用 0 取代掉，希望之後可以找到更省時省力的方法。

```
1 train_df[['BloodPressure', 'SkinThickness', 'Insulin']] = train_df[['BloodPressure', 'SkinThickness', 'Insulin']].replace('No', 0)
2 train_df['BloodPressure'] = train_df['BloodPressure'].replace("Don't Know", 0)
3 train_df['SkinThickness'] = train_df['SkinThickness'].replace("1No", 0)
4 train_df['SkinThickness'] = train_df['SkinThickness'].replace("2No", 0)
5 train_df['SkinThickness'] = train_df['SkinThickness'].replace("3No", 0)
6 train_df['SkinThickness'] = train_df['SkinThickness'].replace("4No", 0)
7 train_df['SkinThickness'] = train_df['SkinThickness'].replace("5No", 0)
8 train_df['SkinThickness'] = train_df['SkinThickness'].replace("4.2cm", 4.2)
9 train_df['SkinThickness'] = train_df['SkinThickness'].replace("3.5cm", 3.5)
10 train_df['Insulin'] = train_df['Insulin'].replace("22No", 0)
11 train_df['Insulin'] = train_df['Insulin'].replace("5No", 0)
12 train_df['Insulin'] = train_df['Insulin'].replace("14No", 0)
13 train_df['Insulin'] = train_df['Insulin'].replace("12No", 0)
14 train_df['Insulin'] = train_df['Insulin'].replace("24No", 0)
15 train_df['Insulin'] = train_df['Insulin'].replace("16No", 0)
16 train_df['Insulin'] = train_df['Insulin'].replace("51No", 0)
17 train_df['Insulin'] = train_df['Insulin'].replace("18No", 0)
18 train_df['Insulin'] = train_df['Insulin'].replace("1No", 0)
19 train_df['Insulin'] = train_df['Insulin'].replace("1No5", 0)
20 train_df['Insulin'] = train_df['Insulin'].replace("4No", 0)
21 train_df['Insulin'] = train_df['Insulin'].replace("1No6", 0)
22 train_df['Insulin'] = train_df['Insulin'].replace("17No", 0)
23 train_df['Insulin'] = train_df['Insulin'].replace("13No", 0)
24 train_df['Insulin'] = train_df['Insulin'].replace("9No", 0)
25 train_df['Insulin'] = train_df['Insulin'].replace("2No4", 0)
26 train_df['Insulin'] = train_df['Insulin'].replace("21No", 0)
27 train_df['Insulin'] = train_df['Insulin'].replace("11No", 0)
28 train_df['Insulin'] = train_df['Insulin'].replace("48No", 0)
29 train_df['Insulin'] = train_df['Insulin'].replace("1No8", 0)
30 train_df['Insulin'] = train_df['Insulin'].replace("3No4", 0)
31 train_df['Insulin'] = train_df['Insulin'].replace("2NoNo", 0)
32 train_df['Insulin'] = train_df['Insulin'].replace("19No", 0)
33 train_df['Insulin'] = train_df['Insulin'].replace("15No", 0)
34 train_df[['BloodPressure', 'SkinThickness', 'Insulin']] = train_df[['BloodPressure', 'SkinThickness', 'Insulin']].astype('float')
```

```
test[['BloodPressure', 'SkinThickness', 'Insulin']] = test[['BloodPressure', 'SkinThickness', 'Insulin']].replace('No', 0)
test['BloodPressure'] = test['BloodPressure'].replace("Don't Know", 0)
test['BloodPressure'] = test['BloodPressure'].replace("4No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("2No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("4No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("1No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("3No", 0)
test['SkinThickness'] = test['SkinThickness'].replace("2.3cm", 2.3)
test['Insulin'] = test['Insulin'].replace("5No", 0)
test['Insulin'] = test['Insulin'].replace("13No", 0)
test['Insulin'] = test['Insulin'].replace("11No", 0)
test['Insulin'] = test['Insulin'].replace("13No", 0)
test['Insulin'] = test['Insulin'].replace("12No", 0)
test['Insulin'] = test['Insulin'].replace("16No", 0)
test['Insulin'] = test['Insulin'].replace("21No", 0)
test['Insulin'] = test['Insulin'].replace("1No5", 0)
test['Insulin'] = test['Insulin'].replace("18No", 0)
test['Insulin'] = test['Insulin'].replace("14No", 0)
test[['BloodPressure', 'SkinThickness', 'Insulin']] = test[['BloodPressure', 'SkinThickness', 'Insulin']].astype('float')
```

結論：這份資料我只做了簡單的清理就丟進模型訓練了，在特徵異常值的地方只清除 BMI 和 BloodPressure 為 0 的部分，因為沒有這方面的背景知識不知道 SkinThickness 這個值為 0 是否正常，但模型的準確率與 MSE 看起來都還可以

就沒有多做調整了，而且最後發現糖尿病可能會與 Glucose、Age、BMI 和 DiabetesPedigreeFunction 較有關。

Reference：

(1)

<https://github.com/ageron/handson->

[ml2/blob/master/02_end_to_end_machine_learning_project.ipynb](https://github.com/ageron/handson-ml2/blob/master/02_end_to_end_machine_learning_project.ipynb)

(2)

<https://chat.openai.com/>

(3)

<https://scikit-learn.org/stable/index.html>